-Introduction to Compiler

1. What is a Compiler?

A **compiler** is a system software that translates a program written in a **high-level programming language** into an equivalent program in a **target language**, usually assembly language or machine code.



2. Why Do We Need a Compiler?

Computer processors understand **machine language**, while programmers prefer high-level languages such as:

- C
- C++
- Java
- Fortran

Machine language is difficult for humans to write and understand.

Therefore, a compiler acts as a **bridge between the programmer and the processor**.

3. Source Program

The program written by the programmer in a high-level programming language is called the **source program** or **source code**.

Example:

```c
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 20;
    printf("%d", a + b);
    return 0;
}
```

This is the input to the compiler.

4. Target Program

The output generated by the compiler is called the **target program**.

It may be:

- Assembly code
- Machine code
- Object code
- Another intermediate/target representation, depending on the compiler design

For example:

Source Code
↓
Compiler
↓
Assembly / Object / Machine Code

9. Compiler vs Interpreter

Both are used to execute programs written in high-level languages, but their processing models differ.

Compiler	Interpreter
Translates a program into a target representation before execution	Generally executes source/translated units incrementally
Can produce a separate target/object/executable representation	Usually does not produce a standalone machine-code executable as its primary operation
Errors are generally reported during compilation	Errors may be reported as execution reaches the problematic part
Compiled code can execute efficiently after compilation	Interpretation generally adds execution overhead
C and C++ commonly use compilation	Python is commonly described as interpreted, though its implementation also uses compilation to bytecode

Difference Between Compiler and Assembler

Basis	Compiler	Assembler
Definition	Converts a **high-level language** program into machine code or assembly code.	Converts an **assembly language** program into machine code.
Input	High-level language like **C, C++, Java**	Assembly language like **MOV, ADD, SUB**
Output	Usually machine/object code	Machine/object code
Level of language	High-level → Low-level	Assembly → Machine language
Translation	More complex because high-level statements need extensive analysis	Relatively simpler because assembly instructions closely correspond to machine instructions
Example	`a = b + c;` → machine/object code	`ADD R1, R2` → machine code
Main work	Performs lexical analysis, syntax analysis, semantic analysis, optimization, code generation, etc.	Converts mnemonics and symbols into corresponding machine instructions.

Errors	Detects syntax, semantic, type, and other compilation errors	Detects assembly syntax, invalid instructions, undefined symbols, etc.

Feature	Compiler	Linker	Loader
Main job	Translates source code into object code	Combines object files and libraries	Loads executable into main memory
Input	Source code	Object files + libraries	Executable file
Output	Object code	Executable file	Program in RAM
Works with	C/C++ source code	Multiple object modules	Operating system & memory
Main function	Translation	Linking	Loading
Example	`main.c → main.o`	`main.o + math.o → program.exe`	`program.exe → RAM`
Errors	Syntax/semantic errors	Undefined references, duplicate symbols	Memory/loading-related errors

12. Important Characteristics of a Compiler

A good compiler generally aims to provide:

1. Correctness

Generated program ko source program ke intended behavior ko preserve karna chahiye.

2. Efficient Target Code

Generated code ko efficient banana.

3. Fast Compilation

Compilation process unnecessarily slow nahi hona chahiye.

4. Good Error Reporting

Errors ko meaningful manner mein report karna.

5. Portability

Compiler design mein portability important ho sakti hai, especially when targeting different machines.

6. Optimization

Program ki performance aur resource usage improve karna.

13. Compiler ke Important Terms

Source Language

Jis language mein original program likha gaya hai.

Example:

C
C++
Java

Source Program

Source language mein written actual program.

Target Language

Jis language mein compiler output produce karta hai.

Example:

Assembly
Machine Language

Target Program

Compiler ke dwara generated output program.

Host Machine

Jis computer/system par compiler **run** ho raha hai.

Target Machine

Jis architecture ke liye compiler **code generate** kar raha hai.

Example:

Compiler runs on:
x86 computer

Compiler generates code for:
ARM processor

14. Overall Working of Compiler

Ek simple example:

x = a + b * c;

Compiler roughly:

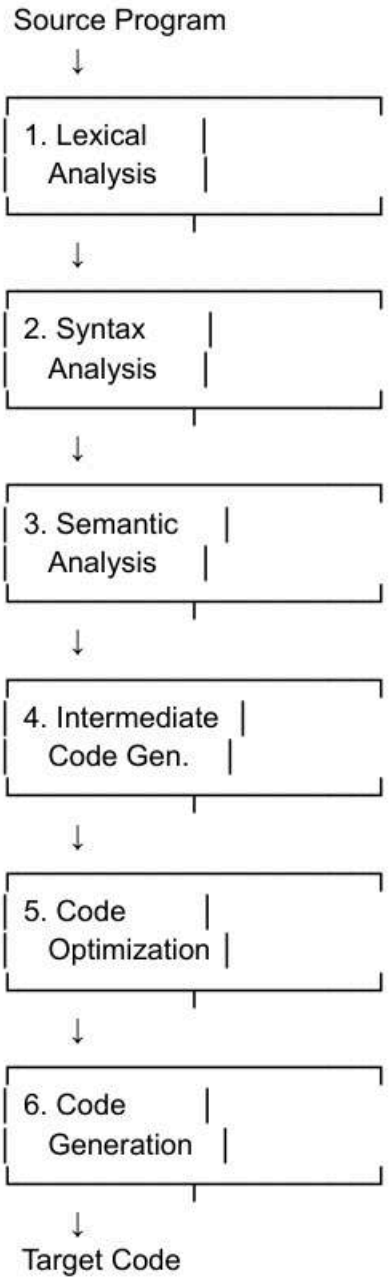
Source Code
↓
Lexical Analysis
↓
Tokens
↓
Syntax Analysis
↓
Parse/Syntax Structure
↓
Semantic Analysis
↓
Intermediate Representation
↓
Optimization
↓
Target Code Generation
↓
Machine/Assembly Code

ii)Phases of a Compiler

A **compiler** converts a high-level language program into machine-level/object code. To do this, it passes through several phases.

Compiler ke major phases hain:

Overall Flow



1. Lexical Analysis

The **lexical analyzer** is the first phase of the compiler.

It reads the source program **character by character** and groups characters into meaningful units called **tokens**.

Example

int sum = a + b;

Tokens are:

int → keyword
sum → identifier
= → operator
a → identifier
+ → operator
b → identifier
; → separator

Main work:

- Removes unnecessary spaces/comments
- Identifies tokens
- Creates/updates the symbol table
- Reports lexical errors

---

2. Syntax Analysis

Syntax analysis is also called **parsing**.

It checks whether the tokens follow the **grammar rules of the programming language**.

Example

Correct:

a = b + c;

Incorrect:

a = + b c;

The parser detects the syntax error.

Output:

A **parse tree** or **syntax tree (AST)**.

---

3. Semantic Analysis

Semantic analysis checks the **meaning of the program**.

Even if the syntax is correct, the program may have a semantic error.

Example

```
int x;
x = "Hello";
```

The syntax may be valid, but assigning a string to an integer variable is a **type-related semantic error**.

Main work:

- Type checking
- Checking variable declarations
- Checking scope
- Checking function arguments
- Checking compatible operations

---

4. Intermediate Code Generation

After semantic analysis, the compiler generates an **Intermediate Representation (IR)**.

It is a representation between source code and machine code.

Example:

```
a = b + c * d;
```

Possible three-address code:

```
t1 = c * d
t2 = b + t1
a = t2
```

Advantage:

The intermediate code makes it easier to generate code for **different target machines**.

5. Code Optimization

The compiler improves the intermediate code so that the final program can execute **more efficiently**.

Example:

```
x = 10 * 2;
```

can be optimized to:

```
x = 20;
```

Some common optimizations:

- Constant folding
- Dead code elimination
- Common subexpression elimination
- Loop optimization
- Strength reduction

6. Code Generation

This is the final major phase.

The compiler converts the optimized intermediate code into **target machine code or assembly code**.

Example:

Intermediate Code
↓
Code Generator
↓
Assembly / Machine Code

It also handles things such as:

- Register allocation
- Instruction selection
- Memory addressing

Supporting Components

A **symbol table** stores information about identifiers used in the program.

For example:

Name	Type	Scope
`x`	int	local
`sum`	function	global
`a`	float	local

Passes of a Compiler

A **pass** is one complete processing of the **source program or intermediate representation** by the compiler.

In simple words:

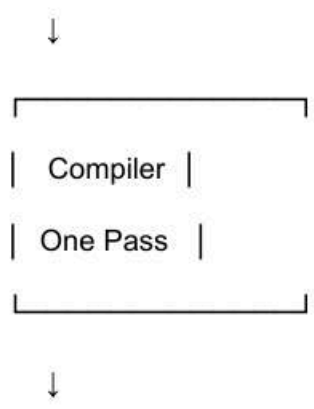
> **Phase = What work is done**
> **Pass = How many times the compiler processes the program**

A compiler can be designed as a **single-pass, two-pass, or multi-pass compiler**.

---

1. Single-Pass Compiler

A **single-pass compiler** processes the source program **only once**.



Example

The compiler reads:

a = b + c;

and performs the required analysis and code generation during one pass.

Advantages

- Faster compilation
- Requires less processing time
- Simple for relatively simple languages

Disadvantages

- Difficult to perform complex optimization
- Difficult to handle forward references

- Less suitable for complex programming languages

---

2. Two-Pass Compiler

A **two-pass compiler** processes the program in **two major passes**.

Source Program

↓

PASS 1

↓

Intermediate Code

↓

PASS 2

↓

Target Code

Pass 1

The compiler performs tasks such as:

- Lexical analysis
- Syntax analysis
- Semantic analysis
- Symbol table construction
- Intermediate representation generation

Pass 2

It performs:

- Code optimization
- Target code generation

  The exact division of phases between passes can vary depending on compiler design.

Advantages

- Better than a single-pass compiler for complex languages
- Allows more analysis and optimization
- Easier to handle forward references

Disadvantages

- Slower than a single-pass compiler
- Requires intermediate storage

---

3. Multi-Pass Compiler

A **multi-pass compiler** processes the program or its intermediate representation **more than two times**.

Source

↓

Pass 1

↓

IR

↓

Pass 2

↓

Optimized IR

↓

Pass 3

↓

Target Code

Each pass can perform a particular task.

For example:

Pass 1 → Lexical/Syntax processing

Pass 2 → Semantic analysis

Pass 3 → Intermediate code generation

Pass 4 → Optimization

Pass 5 → Code generation

Advantages

- Allows sophisticated optimization
- Easier to design and maintain
- Suitable for complex programming languages
- Better separation of compiler tasks

Disadvantages

- Takes more compilation time
- Requires more memory/storage
- Compiler design can be more complex

Phase vs Pass — Very Important ⭐

Phase	Pass
Describes **what task** the compiler performs	Describes **how many times** the compiler processes the program
Examples: lexical analysis, syntax analysis	Single-pass, two-pass, multi-pass
Logical division	Processing/implementation division
Several phases can be combined into one pass	One pass can perform multiple phases

Bootstrapping in Compiler Design

Bootstrapping is the process of **developing a compiler for a programming language using the same language itself**.

In simple words:

> **A compiler is written in the language that it is designed to compile.**

For example, suppose we want to create a compiler for a new language called **X**. If the compiler for X is itself written in X, we use a previously available compiler/interpreter or another technique to compile the initial version. Later, the new X compiler can compile its own source code.

Why is Bootstrapping Needed?

When a new programming language is created, there is initially **no compiler available for that language**.

So a problem occurs:

We need an X compiler

↓

But the X compiler is written in X

↓

How do we compile the X compiler?

Bootstrapping provides a way to solve this **initial compiler development problem**.

How Bootstrapping Works

Consider a language **X**.

Step 1: Create a simple compiler

Initially, create a basic compiler for X using an **existing language**, such as C.

Compiler for X

written in C

↓

Existing C Compiler

↓

X Compiler

Step 2: Rewrite the compiler in X

Once the basic X compiler exists, developers can rewrite the compiler using **X itself**.

X Compiler Source

(written in X)

↓

Existing X Compiler

↓

New X Compiler

Step 3: Self-compilation

The new X compiler can then compile its own source code.

X Compiler Source

↓

X Compiler

↓

X Compiler

This process is called **self-hosting**.

Example

Suppose we are developing a language called **MyLang**.

Initially:

MyLang Compiler → written in C

↓

C Compiler

↓

MyLang Compiler

After development:

MyLang Compiler → written in MyLang

↓

Existing MyLang Compiler

↓

New MyLang Compiler

Eventually, the compiler becomes **self-hosting**.

Advantages of Bootstrapping

1. **Self-hosting** – The compiler can compile its own source code.
2. **Improves language development** – Developers use the language while developing its compiler.
3. **Easier maintenance** – Compiler developers can work in the same language.
4. **Helps portability** – A compiler can be adapted to different machines/architectures.
5. **Tests the language** – Writing a compiler in the language helps reveal weaknesses or limitations of the language.

Bootstrapping vs Cross-Compilation

Don't confuse these two:

Bootstrapping:

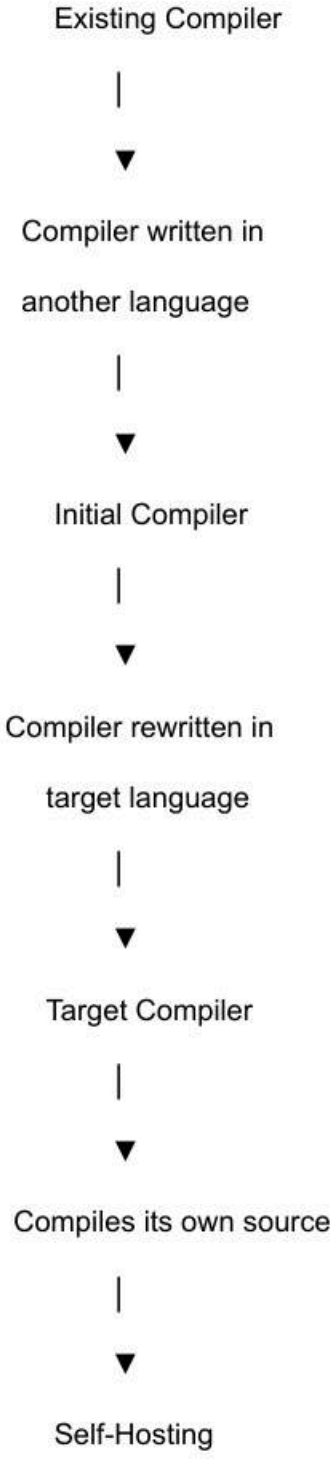
> Building a compiler, eventually allowing it to compile **itself**.

Cross-compilation:

> A compiler running on one machine produces code for **another machine/platform**

5. Bootstrapping Diagram

Exam mein ye diagram bana sakte ho:



1. Finite State Machine (FSM)

A **Finite State Machine (FSM)** is a mathematical model used to represent a system that has a **finite number of states** and changes from one state to another based on the input it receives.

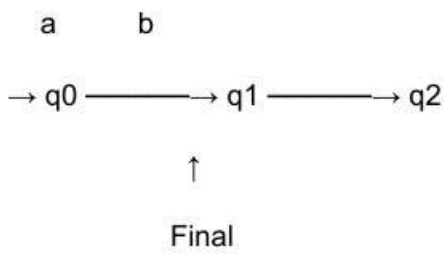
Main Components of FSM

An FSM generally contains:

1. **States** – Different conditions of the machine.
2. **Input symbols** – Symbols given to the machine.
3. **Transitions** – Rules that specify how the machine moves between states.
4. **Initial state** – The state where the machine starts.
5. **Final/Accepting states** – States that indicate successful recognition.

Example

Suppose we want to recognize the string `"ab"`.



- `q0` = starting state
- Reading `a` → move to `q1`
- Reading `b` → move to `q2`
- `q2` = accepting state

If input is `ab`, the string is accepted.

Formal Definition

An FSM can be represented as:

M = (Q, Σ, δ, q_0, F)

Where:

- `Q` = finite set of states
- `Σ` = input alphabet
- `δ` = transition function
- q_0 = initial state
- `F` = set of final states

---

2. Types of Finite State Machines

In compiler design, the two important types are:

A. DFA — Deterministic Finite Automaton

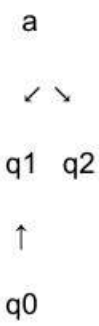
For every state and input symbol, there is **exactly one possible transition**.

q0 --a--> q1

If the machine is in `q0` and receives `a`, it must go to `q1`.

B. NFA — Non-Deterministic Finite Automaton

For a state and input, there can be **zero, one, or multiple possible transitions**.



The same input can lead to multiple states.

NFA can also have **ε-transitions**, where the machine changes state without consuming an input symbol.

3. DFA vs NFA

Feature	DFA	NFA
Full form	Deterministic Finite Automaton	Non-Deterministic Finite Automaton
Transition	Exactly one transition	0, 1, or multiple transitions
ε-transition	Not allowed	Allowed
Implementation	Easier to execute	More complex to execute directly

Number of possible paths	One	Multiple
Power	Same as NFA	Same as DFA
Used in lexical analysis	Very common	Often converted to DFA

Important Point ⭐

DFA and NFA have the same expressive power.

Both recognize **regular languages**.

A common process is:

Regular Expression

↓

NFA

↓

DFA

↓

Minimized DFA

↓

Lexical Analyzer

---

4. Regular Expression

A **Regular Expression (RE)** is a notation used to describe a **set of strings/patterns** belonging to a regular language.

In compiler design, regular expressions are mainly used to **specify the patterns of tokens**.

For example:

Identifier → letter(letter | digit)*

This describes identifiers such as:

x

sum

abc123

student1

---

5. Basic Operators of Regular Expression

There are three fundamental operators.

1. Union / Alternation |

Means **OR**.

a | b

Means either:

a

or

b

---

2. Concatenation

Putting expressions next to each other means they occur **in sequence**.

ab

Accepts:

ab

It does not accept a or b individually.

---

3. Kleene Star *

Means **zero or more repetitions**.

a*

Accepts:

ε

a

aa

aaa

aaaa

...

Here ε means the **empty string**.

Other commonly used operators

+ → one or more repetitions

a+

Accepts:

a

aa

aaa

...

? → zero or one occurrence

a?

Accepts:

ε

a

---

6. Regular Expression Examples in Compiler

Regular expressions are used to describe different tokens.

Identifier

letter(letter | digit)*

Examples:

x

sum

total1

student123

Integer

digit+

Examples:

10

25

1000

Decimal Number

A simplified pattern:

digit+ . digit+

Examples:

10.5

3.14

99.99

Whitespace

(space | tab | newline)+

Relational Operators

< | > | <= | >= | == | !=

Keyword

A keyword can be represented by its exact character sequence.

For example:

if

recognizes the keyword `if`.

---

7. Relationship Between Regular Expressions and FSM

This is **very important for exams**.

Regular Expressions and FSM are two different ways of describing **regular languages**.

Regular Expression

↓

NFA / FSM

↓

DFA

A regular expression describes **what pattern should be recognized**, while an FSM describes **how that pattern can be recognized step-by-step**.

Example

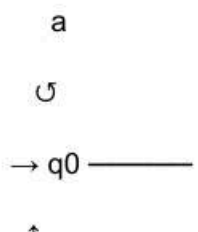
Regular expression:

a*

means:

Zero or more as.

An FSM can recognize it:



`q0` is both the starting and accepting state.

So:

> **Regular Expression = Pattern/Specification**
> **FSM = Recognition Mechanism**

---

8. Application of Regular Expressions to Lexical Analysis

Lexical analysis is the first phase of a compiler. It converts source code into **tokens**.

Regular expressions are used to **define the patterns of these tokens**.

Example Source Code

int total = a + 10;

The lexical analyzer needs to identify:

int → Keyword

total → Identifier

= → Operator

a → Identifier

+ → Operator

10 → Number

; → Separator

Regular expressions can define these token patterns.

For example:

Identifier → letter(letter | digit)*

Number → digit+

The lexical analyzer then uses these patterns to recognize tokens.

Typical Process

Regular Expressions

↓

NFA

↓

DFA

↓

Lexical Analyzer

↓

Tokens

---

9. Role of FSM in Lexical Analysis

FSM plays a major role in **recognizing tokens**.

The lexical analyzer reads the source program character by character and uses an FSM to determine **which token the characters form**.

Example: Recognizing an Identifier

Suppose:

letter = a-z, A-Z

digit = 0-9

Identifier pattern:

letter(letter | digit)*

For:

abc123

FSM processes:

a → b → c → 1 → 2 → 3

and finally reaches an **accepting state**.

Therefore:

abc123 → IDENTIFIER

FSM helps recognize:

- Keywords
- Identifiers
- Numbers
- Operators
- Delimiters
- Whitespace
- Comments
- Other token patterns

🔥 Complete Connection — Very Important

You should understand these three together:

Regular Expression

↓

Describes token pattern

↓

NFA

↓

Convert to DFA

↓

DFA/FSM recognizes

↓

Lexical Analyzer

↓

Tokens

Example

For identifier:

Regular Expression:

letter(letter | digit)*

↓

FSM/DFA recognizes:

abc123

↓

Lexical analyzer produces:

<IDENTIFIER, abc123>

⭐ Exam Revision Table

Topic	One-line meaning
FSM	Machine with finite states used to recognize patterns
DFA	Exactly one transition for each state/input
NFA	Can have multiple/no transitions and ε-transitions

Regular Expression	Describes a regular pattern/language
**Union `	`**
Concatenation	Sequence
Kleene Star *	Zero or more repetitions
RE → FSM	Converts a pattern specification into a recognizer
Lexical Analysis	Converts source characters into tokens
FSM in Lexical Analysis	Recognizes token patterns

Optimization of DFA-Based Pattern Matchers

1. What is a DFA-Based Pattern Matcher?

A **DFA-based pattern matcher** is a mechanism that uses a **Deterministic Finite Automaton (DFA)** to recognize patterns in input text.

In compiler design, it is mainly used in the **lexical analyzer** to recognize tokens such as:

- Keywords
- Identifiers
- Numbers
- Operators
- Whitespace
- Comments

The basic process is:

Regular Expression
↓
NFA
↓
DFA
↓
Optimization
↓
DFA-based Pattern Matcher
↓
Tokens

The problem is that a DFA can contain many states and transitions. Therefore, we optimize it to make **token recognition faster and use less memory**.

---

2. What Does Optimization Mean?

Optimization of a DFA-based pattern matcher means modifying the DFA implementation so that it:

- Executes faster
- Uses less memory
- Requires fewer states or transitions
- Performs fewer operations while scanning input
- Maintains exactly the **same language/pattern recognition behavior**

Important ⭐

Optimization **must not change what strings are accepted**.

For example, if the original DFA recognizes:

digit+

then the optimized DFA must recognize exactly the same set of numbers.

---

3. Why Do We Need DFA Optimization?

A lexical analyzer processes **every character of the source program**.

For a large program, millions of characters may need to be processed.

If the DFA is large:

Input character
↓
Find current state
↓
Find transition
↓
Move to next state
↓
Repeat...

Even a small improvement in each character's processing can significantly improve compiler performance.

Therefore, DFA optimization focuses mainly on:

1. Reducing number of states

2. Reducing memory used by transitions

3. Making transition lookup faster

4. Removing unnecessary computations

---

4. Major Techniques for DFA Optimization

There are several techniques used to optimize DFA-based pattern matchers.

A. DFA State Minimization

This is one of the most important techniques.

Definition

DFA minimization is the process of reducing the number of states in a DFA while preserving its language-recognition capability.

Suppose an original DFA has:

10 states

After minimization:

6 states

The new DFA accepts **exactly the same strings**.

Original DFA
10 states
↓
State Minimization
↓
Optimized DFA
6 states

Why?

Sometimes two states behave identically for all possible future inputs.

Such states are called **equivalent states** and can be merged.

B. Removing Unreachable States

A state that can **never be reached from the initial state** is unnecessary.

Example:

a
→ q0 ----> q1

q2 q3

If there is no path from `q0` to `q2` or `q3`, they are unreachable.

So they can be removed:

→ q0 ----> q1

Benefit

- Fewer states
- Less memory
- Faster DFA processing

---

5. Merging Equivalent States

Two states are **equivalent** if they behave identically for every possible remaining input string.

Suppose:

q1 → q2 on 'a'
q1 → q3 on 'b'

q4 → q2 on 'a'
q4 → q3 on 'b'

If q1 and q4 also have the same accepting/non-accepting status, they may be equivalent.

Therefore:

q1 + q4
↓
One state

This reduces the size of the DFA.

---

6. Transition Table Optimization

A DFA can be represented using a **transition table**.

For example:

State	a	b	digit
q0	q1	q2	q3
q1	q1	q2	q3
q2	q4	q2	q3

For a large alphabet, this table can become very large.

Optimization techniques can reduce the storage required for these transitions.

---

7. Character Classes

Many input characters behave identically.

For example, while recognizing an identifier:

a-z
A-Z
0-9
_

Instead of treating every character separately, we can group them into **character classes**.

For example:

LETTER
DIGIT
UNDERSCORE
OTHER

Then:

'a', 'b', 'c', ... 'z'

can belong to the same `LETTER` class.

Benefit

Instead of storing transitions for every individual character, we can store transitions for groups.

This reduces:

- Memory
- Transition-table size
- Lookup complexity

---

8. Default Transitions

Sometimes many characters from a state lead to the **same destination state**.

Instead of storing every transition individually, we can store a **default transition**.

For example:

q0:
'a' → q1
'b' → q1

'c' → q1
'd' → q1
...
'z' → q1

Instead of storing 26 separate entries, we can represent them as:

LETTER → q1

This is especially useful when character classes are used.

---

9. Efficient Transition Lookup

The lexical analyzer must repeatedly perform:

next_state = transition[current_state][input_character]

If the transition table is large, lookup can become expensive.

Therefore, implementations can use efficient data structures such as:

- Arrays
- Compressed tables
- Character-class tables
- Sparse representations

The goal is to find the next state quickly.

---

10. Transition Table Compression

A complete DFA transition table can consume a lot of memory.

For example:

100 states × 256 characters

would require:

25,600 transition entries

Many of those entries may be identical or unnecessary.

Transition-table compression stores only the information that is actually needed.

Example

Instead of:

q0 → 256 entries
q1 → 256 entries
q2 → 256 entries
...

we can store common/default transitions compactly.

Benefit

> **Less memory + faster cache usage**

---

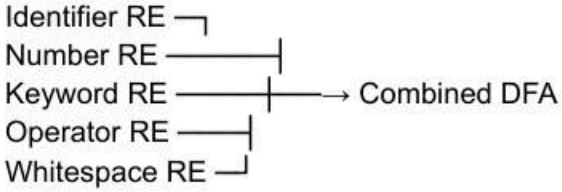
11. Combining Multiple DFAs

In lexical analysis, we usually need to recognize **many token types**.

For example:

Identifier
Integer
Keyword
Operator
Whitespace

Instead of running a separate DFA for each token, the lexical analyzer can combine patterns into a **single DFA**.



The combined DFA can recognize different token types in one scanning process.

This avoids repeatedly restarting different pattern matchers.

---

12. Dead-State Handling

A **dead state** is a state from which no accepting state can ever be reached.

For example:

q0 → q1 → qDead
↺

Once the DFA enters `qDead`, it cannot accept the input.

The implementation can handle such states efficiently rather than performing unnecessary processing.

---

13. Using Accepting-State Information Efficiently

In lexical analysis, the DFA must know when a token has been recognized.

For example:

identifier:
letter(letter | digit)*

When the DFA reaches an accepting state, it records:

TOKEN = IDENTIFIER

An optimized matcher can store accepting-state information efficiently, such as:

State → Token Type

Example:

q5 → IDENTIFIER
q8 → INTEGER
q12 → PLUS

This allows quick token identification.

---

14. Example of Optimization

Suppose we have a DFA for:

digit+

Initially:

q0 --digit--> q1
q1 --digit--> q1
q2
q3

Suppose q2 and q3 are unreachable.

Remove them:

q0 --digit--> q1
q1 --digit--> q1

Now the DFA has fewer states and requires less memory.

---

15. Optimization Goals

The main goals can be summarized as:

Goal	Meaning
Reduce states	Remove unnecessary/equivalent states
Reduce memory	Compress transition tables
Fast lookup	Find next state quickly
Reduce operations	Avoid unnecessary calculations
Improve cache usage	Keep frequently accessed data compact
Preserve correctness	Recognize exactly the same language

---

16. Optimized DFA Matcher in Lexical Analysis

The complete process becomes:



Implementation of Lexical Analysers

A **lexical analyzer** is the first phase of a compiler. It reads the source program **character by character** and converts it into a sequence of **tokens**.

The implementation of a lexical analyzer means **how we actually build the lexer** so that it can recognize tokens efficiently.

---

1. Basic Working of a Lexical Analyzer

Consider this C statement:

int sum = a + 10;

The lexical analyzer reads it and produces:

<int, keyword>
<sum, identifier>
<=, operator>
<a, identifier>
<+, operator>
<10, number>
<;, separator>

Overall process

Source Program
↓
Lexical Analyzer
↓
Characters → Tokens
↓
Parser

---

2. Main Components

A lexical analyzer generally contains:

1. Input Buffer

Stores the characters of the source program so they can be read efficiently.

2. Lexical Rules

Rules that define what constitutes a token.

For example:

Identifier → letter(letter | digit)*
Number → digit+

3. Pattern Matcher

Recognizes whether the input characters match a token pattern.

It can use:

- Regular expressions
- DFA
- NFA

4. Token Generator

Creates a token after recognizing a lexeme.

5. Symbol Table

Stores information about identifiers.

6. Error Handler

Reports invalid characters or invalid token patterns.

---

3. Methods of Implementing Lexical Analyzers

There are several common approaches.

A. Hand-Written Lexical Analyzer

A lexer can be written manually using a programming language such as C, C++, Java, or Python.

The programmer explicitly writes rules for recognizing tokens.

Example

```
if (isalpha(ch)) {
   // recognize identifier
}
else if (isdigit(ch)) {
   // recognize number
}
```

Advantages

- Simple for small languages
- Complete control over implementation
- Easy to customize

Disadvantages

- Time-consuming for large languages
- More chances of programming errors
- Maintaining many token rules can become difficult

---

4. Lexical Analyzer Using Regular Expressions

Token patterns can first be specified using **regular expressions**.

Example:

Identifier → letter(letter | digit)*
Integer → digit+

Then these patterns can be converted into automata.

Regular Expression
↓
NFA
↓
DFA
↓
Lexical Analyzer

This approach provides a systematic way to build a lexer.

---

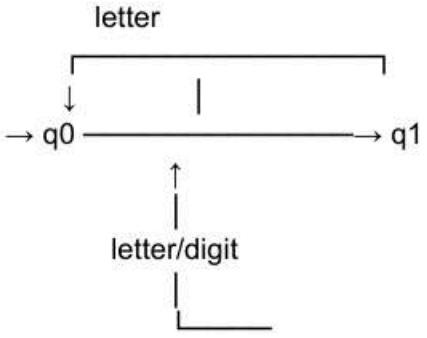
5. Lexical Analyzer Using DFA

A **DFA** can directly recognize token patterns.

For example, identifier:

letter(letter | digit)*

A simplified DFA:



- `q0` = start state
- `q1` = accepting state
- First character must be a letter
- Following characters can be letters or digits

For:

student123

the DFA reaches the accepting state, so the lexer identifies it as:

IDENTIFIER

---

6. Lexical Analyzer Generator

Instead of manually writing the entire lexical analyzer, we can use a **lexical analyzer generator**.

A lexical analyzer generator takes:

Token patterns + actions
↓
Lexer Generator
↓
Lexical Analyzer

A well-known example is **LEX/Flex**.

---

7. LEX

LEX is a tool used to automatically generate a lexical analyzer.

The programmer provides:

- Regular expressions
- Corresponding actions

LEX generates a lexer, traditionally in C.

Basic structure of a Lex program

Definitions
%%
Rules
%%
User Subroutines

Example

```
%%
[0-9]+      { printf("NUMBER"); }
[a-zA-Z]+   { printf("IDENTIFIER"); }
[ \t\n]+    { /* ignore whitespace */ }
%%
```

Here:

[0-9]+

matches one or more digits.

And:

[a-zA-Z]+

matches one or more letters.

---

8. Input Buffering

Reading one character directly from the source file every time can be slow.

Therefore, lexical analyzers use **input buffering**.

Instead of:

Disk → Character → Lexer
Disk → Character → Lexer
Disk → Character → Lexer

a block of characters is read into memory:

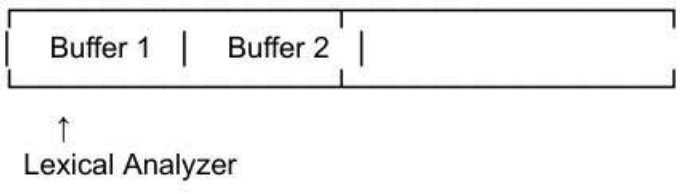
Disk
↓
Input Buffer
↓
Lexical Analyzer

This improves performance.

---

9. Two-Buffer Scheme

A traditional technique is the **two-buffer scheme**.



When one buffer is exhausted, the other buffer is used while the first buffer is refilled.

This reduces the number of input operations.

---

10. Lexeme Recognition

A **lexeme** is the actual sequence of characters that forms a token.

Example:

int total = 25;

Lexeme	Token
`int`	Keyword
`total`	Identifier
`=`	Assignment operator
`25`	Number
`;`	Separator

The lexer recognizes each lexeme and generates the corresponding token.

---

11. Lookahead

Sometimes the lexer needs to look at the **next character** before deciding which token it has.

Example:

<
<=

If the lexer sees `<`, it checks the next character.

< + next character

If the next character is `=`:

<= → LE

Otherwise:

< → LESS_THAN

This is called **lookahead**.

---

12. Retracting

Sometimes the lexer reads one character too far.

Example:

abc123+

While recognizing:

abc123

the lexer reads + to determine whether the identifier has ended.

Since + belongs to the next token, the lexer must **retract** or put it back for the next token.

```
abc123 | +
       ↑
  next token
```

---

13. Symbol Table Interaction

When an identifier is found, the lexical analyzer may enter it into the **symbol table**.

Example:

int student;

The lexer recognizes:

student → IDENTIFIER

and information about `student` can be stored in the symbol table.

Later compiler phases can use that information.

---

14. Error Handling

The lexical analyzer must detect invalid character sequences.

Example:

int @name;

If `@` is not a valid character in the language's identifier/token rules, the lexer reports a **lexical error**.

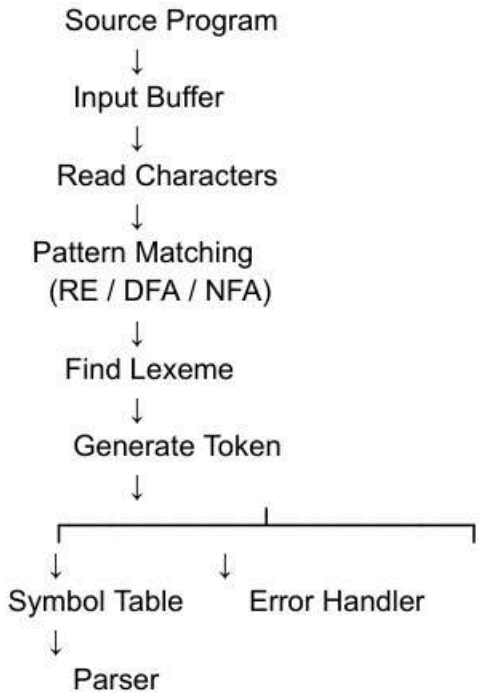
Typical lexical errors include:

- Invalid character
- Invalid number format
- Unterminated string
- Unterminated comment
- Invalid identifier

---

15. Complete Implementation Process

The complete implementation can be understood as:



⭐ Important Exam Point

There are two major ways to implement a lexical analyzer:

1. Manual Implementation

Programmer writes the lexer directly using a programming language.

Token Rules
↓
Programmer
↓
Hand-written Lexer

2. Lexical Analyzer Generator

Use tools such as **LEX/Flex**.

Regular Expressions
+
Actions
↓
LEX/Flex
↓
Generated Lexical Analyzer

1. Lexical Analyzer Generator

A **Lexical Analyzer Generator** is a software tool that automatically generates a **lexical analyzer (lexer/scanner)** from a set of **regular expressions and associated actions**.

Instead of manually programming a lexer for every token, we specify the token patterns, and the generator creates the lexer.

Basic idea

Regular Expressions
+
Actions
↓
Lexical Analyzer Generator
↓
Lexical Analyzer

↓
Tokens

Example

Suppose we want to recognize:

NUMBER → digit+
IDENTIFIER → letter(letter | digit)*

We provide these rules to the generator.

It produces a program that can recognize:

123 → NUMBER
student1 → IDENTIFIER

Why do we need a Lexical Analyzer Generator?

Writing a lexical analyzer manually can be difficult when a programming language contains hundreds of token patterns.

A generator:

- Reduces programming effort
- Automatically creates scanner code
- Uses regular expressions for token specification
- Produces efficient pattern matching code
- Makes modifications easier

Important Point ⭐

Lexical analyzers are generally based on **regular languages**, so **regular expressions and finite automata** are commonly used to specify and recognize tokens.

---

2. LEX Compiler

LEX is a **lexical analyzer generator**. It takes a specification containing regular expressions and actions and generates a lexical analyzer, traditionally as C source code.

The generated scanner is commonly called `yylex()` in traditional Lex-style systems.

Working

LEX Specification
↓
LEX
↓
Generated C Scanner
↓
C Compiler
↓
Executable Lexer

Structure of a LEX Program

A Lex specification generally has **three sections**:

Definitions
%%
Rules
%%
User Subroutines

1. Definitions

Contains declarations, definitions, and optional macros.

2. Rules

Contains:

pattern action

For example:

```
[0-9]+      { printf("NUMBER"); }
[a-zA-Z]+   { printf("IDENTIFIER"); }
```

3. User Subroutines

Contains supporting C code and functions.

Example

A simplified Lex specification:

```
%%
[0-9]+      { printf("NUMBER"); }
```

```
[a-zA-Z]+     { printf("WORD"); }
[ \t\n]+      { /* ignore whitespace */ }
%%
```

If the input is:

hello 123

the lexer can recognize:

hello → WORD
123 → NUMBER

Advantages of LEX

1. Automatically generates a lexical analyzer.
2. Easy to specify token patterns.
3. Reduces manual coding.
4. Regular expressions make token definitions concise.
5. Works well with parser generators such as YACC.

---

3. Formal Grammars

A **formal grammar** is a mathematical system used to describe the **syntax/structure of strings in a language**.

A grammar defines which sequences of symbols are valid according to a set of production rules.

A grammar is commonly represented as:

G = (V, T, P, S)

Where:

- **V** = set of non-terminals
- **T** = set of terminals
- **P** = production rules
- **S** = start symbol

Example Grammar

Consider:

E → E + T
E → T
T → id

Here:

- `E`, `T` → non-terminals
- `+`, `id` → terminals
- `E` → start symbol
- Production rules describe valid expressions.

---

4. Application of Formal Grammars to Syntax Analysis

The **syntax analyzer (parser)** checks whether the sequence of tokens produced by the lexical analyzer follows the grammar of the programming language.

Source Code
↓
Lexical Analyzer
↓
Tokens
↓
Syntax Analyzer
↓
Parse Tree / AST

For example:

a + b

The lexer produces:

id + id

A grammar can define:

E → E + E
E → id

The parser checks whether:

id + id

is valid according to the grammar.

---

5. Notation Used in Formal Grammars

Several symbols are commonly used.

Terminal

A symbol that appears in the final generated string.

Example:

id, +, *, (,)

Non-terminal

A symbol used to represent a syntactic category.

Example:

E, T, F

Production

A rule describing how a non-terminal can be replaced.

Example:

$E \rightarrow E + T$

Start Symbol

The non-terminal from which derivation begins.

Example:

S

or:

E

---

6. Ambiguity

A grammar is called **ambiguous** if at least one valid string in the language has **more than one parse tree** or more than one leftmost/rightmost derivation.

Example

Consider:

$E \rightarrow E + E$
$E \rightarrow E * E$
$E \rightarrow id$

Now consider:

id + id * id

There can be two interpretations:

(id + id) * id

or:

id + (id * id)

Therefore, this grammar is ambiguous.

Why is Ambiguity a Problem?

The compiler needs to know exactly how an expression should be interpreted.

For example:

a + b * c

Normally multiplication has higher precedence than addition:

a + (b * c)

A grammar that does not properly specify precedence can lead to multiple parse trees.

Removing Ambiguity

We can design a grammar that explicitly represents precedence.

For example:

E → E + T | T
T → T * F | F
F → id

Here:

* has higher precedence than +

because multiplication is handled at the T level.

---

7. YACC

YACC stands for:

> **Yet Another Compiler-Compiler**

YACC is a **parser generator** used to automatically generate a **syntax analyzer/parser** from a formal grammar, traditionally producing C code.

While LEX generates the **lexical analyzer**, YACC generates the **parser**.

Basic relationship

Regular Expressions
↓
LEX
↓
Lexical Analyzer
↓
Tokens
↓
YACC
↓
Parser
↓
Parse Tree / AST

What does YACC take as input?

YACC generally takes:

- Context-Free Grammar (CFG)
- Production rules
- Semantic actions

- Token declarations

and generates parser code.

Example Grammar

A simple expression grammar might contain:

```
E : E '+' T
  | T
  ;

T : T '*' F
  | F
  ;

F : ID
  ;
```

YACC uses such grammar rules to generate a parser.

---

8. LEX vs YACC

Feature	LEX	YACC
Full form	Lexical Analyzer Generator	Yet Another Compiler-Compiler
Main purpose	Lexical analysis	Syntax analysis
Input	Regular expressions	Context-Free Grammar
Produces	Lexer	Parser
Recognizes	Tokens	Syntactic structures
Based mainly on	Regular languages / finite automata	Context-free grammars / parsing algorithms
Example	Identifier, number, keyword	Expression, statement, block

Easy trick:

LEX → Tokens
YACC → Grammar/Parser

---

9. Syntactic Specification of Programming Languages

Syntactic specification describes the rules that determine how tokens can be arranged to form valid programs.

For example, a language might specify:

statement → identifier = expression ;

This tells us that an assignment statement consists of:

identifier + = + expression + ;

Why is Syntax Specification Needed?

It tells the compiler:

- Which programs are syntactically valid
- How statements are structured
- How expressions are formed
- How operators are arranged
- How blocks and declarations are formed

Syntax is generally specified using a **Context-Free Grammar (CFG)** or a notation based on CFGs such as **BNF**.

---

10. Context-Free Grammar (CFG)

A **Context-Free Grammar** is a formal grammar in which every production has a **single non-terminal on the left-hand side**.

General form:

A → α

Where:

- A = one non-terminal
- α = sequence of terminals and/or non-terminals

A CFG is represented by:

G = (V, T, P, S)

Example

Consider:

E → E + T
E → T
T → id

This grammar describes simple arithmetic expressions.

For:

id + id

we can derive:

E
↓
E + T
↓
T + T
↓
id + id

---

11. Derivation

Definition

Derivation is the process of applying production rules of a grammar to generate a string from the start symbol.

Suppose:

E → E + T
E → T
T → id

To generate:

id + id

Start with:

E

Apply:

$E \rightarrow E + T$

We get:

$E + T$

Now:

$E \rightarrow T$

gives:

$T + T$

Then:

$T \rightarrow id$

and:

$T \rightarrow id$

giving:

$id + id$

Therefore:

$E \Rightarrow E + T \Rightarrow T + T \Rightarrow id + id$

This is called a **derivation**.

---

12. Leftmost Derivation

In **leftmost derivation**, we always replace the **leftmost non-terminal first**.

Example:

$E \rightarrow E + T$
$E \rightarrow T$
$T \rightarrow id$

For:

id + id

we have:

E
⇒ E + T
⇒ T + T
⇒ id + T
⇒ id + id

The leftmost non-terminal is expanded at each step.

---

13. Rightmost Derivation

In **rightmost derivation**, we always replace the **rightmost non-terminal first**.

Example:

E
⇒ E + T
⇒ E + id
⇒ T + id
⇒ id + id

The rightmost non-terminal is expanded first.

Important ⭐

Both leftmost and rightmost derivations can generate the **same final string**.

---

14. Parse Tree

A **parse tree** is a tree representation of how a string is derived from a grammar.

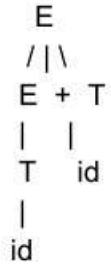
Consider:

E → E + T
E → T
T → id

For:

id + id

the parse tree is:



Parts of Parse Tree

- **Root** → Start symbol
- **Internal nodes** → Non-terminals
- **Leaf nodes** → Terminals
- Reading leaves from left to right gives the generated string.

---

15. Capabilities of CFG

CFGs are very important in compiler design because they can describe many structures that regular expressions cannot conveniently describe.

1. Nested Structures

CFG can represent nested constructs.

Example:

((a))

A grammar can describe balanced parentheses:

S → (S)
S → SS
S → ε

---

2. Arithmetic Expressions

CFG can describe expressions such as:

a + b * c
(a + b) * c
a * (b + c)

---

3. Statements

CFG can describe programming-language statements:

```
if (condition)
   statement
```

or:

```
while (condition)
   statement
```

---

4. Blocks

CFG can represent blocks containing multiple statements:

```
{
   statement
   statement
   statement
}
```

---

5. Operator Precedence

A properly designed CFG can represent precedence between operators.

For example:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid id$$

Here * has higher precedence than +.

---

6. Recursion

CFGs naturally support recursive structures.

Example:

$$E \rightarrow E + T$$

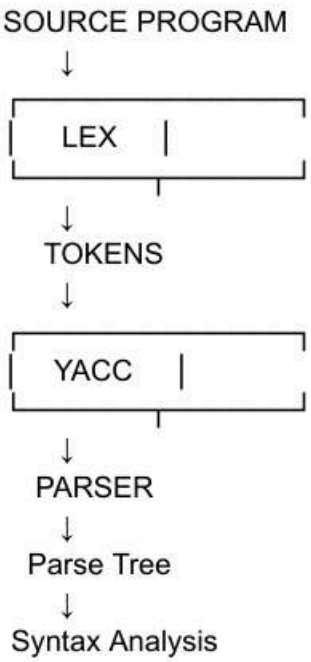
The non-terminal E appears again on the right side.

This is useful for representing:

- Nested expressions
- Function calls
- Blocks
- Statements
- Lists

🔥 Complete Picture

You should connect all these topics like this:



And internally:

LEX:
Regular Expressions
↓
DFA
↓
Tokens

YACC:
Context-Free Grammar
↓
Parser
↓
Parse Tree

⭐ Quick Revision

Topic	Key Point
Lexical Analyzer Generator	Automatically generates a lexer from token patterns
LEX	Generates lexical analyzer
Formal Grammar	Mathematical description of language syntax
Ambiguity	One string has more than one parse tree/derivation
YACC	Generates a parser from grammar
Syntactic Specification	Defines valid arrangement of tokens
CFG	Grammar with one non-terminal on LHS of each production
Derivation	Applying grammar rules to generate a string
Leftmost Derivation	Expand leftmost non-terminal first
Rightmost Derivation	Expand rightmost non-terminal first
Parse Tree	Tree representation of a derivation
CFG Capabilities	Nesting, recursion, expressions, statements, blocks, etc.

Unit 2: Basic Parsing Techniques

Parsing is a major part of **Syntax Analysis** in Compiler Design. The parser receives **tokens from the lexical analyzer** and checks whether those tokens follow the grammar of the programming language.

Basic flow

Source Program
↓
Lexical Analyzer
↓
Tokens
↓
Parser
↓
Parse Tree / Syntax Tree
↓
Semantic Analysis

We will cover these topics one by one:

1. **Parsers**
2. **Shift-Reduce Parsing**
3. **Operator Precedence Parsing**
4. **Top-Down Parsing**

---

1. Parser

A **parser** is a component of the compiler that performs **syntax analysis**. It takes tokens generated by the lexical analyzer and checks whether they form a valid structure according to the **context-free grammar (CFG)** of the programming language.

In simple words:

> **Parser checks whether the sequence of tokens follows the grammatical rules of the programming language.**

Example

Suppose the input is:

a + b * c

The lexical analyzer produces:

id + id * id

The parser checks whether this sequence is valid according to a grammar such as:

E → E + T | T
T → T * F | F
F → id

The parser can construct a parse tree representing:

```
  +
 / \
a   *
   / \
  b   c
```

This also shows that * has higher precedence than +.

Input and Output of Parser

Input

The parser receives a sequence of **tokens**:

id + id * id

Output

If the sequence is valid:

Parse Tree / Syntax Tree

If invalid:

Syntax Error

Example of invalid input

a + * b

Tokens:

id + * id

This does not follow the grammar, so the parser reports a syntax error.

Functions of a Parser

A parser performs several important tasks.

1. Checks syntax

It determines whether the input follows the grammar.

a + b → Valid
a + * b → Invalid

2. Constructs parse tree

The parser can construct a **parse tree** or **abstract syntax tree (AST)**.

3. Reports syntax errors

For example:

a + b *

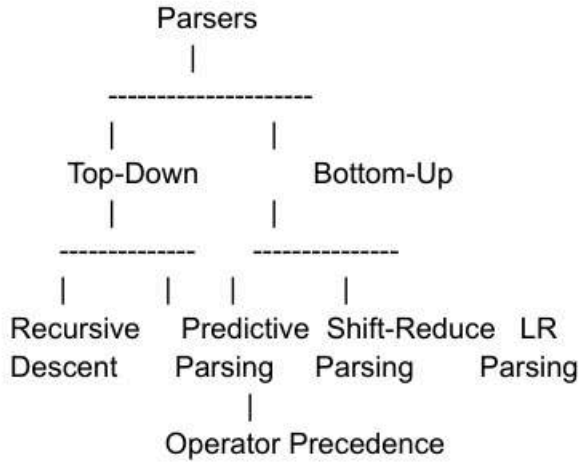
The parser can identify that an operand is missing after *.

4. Supports semantic analysis

The parse tree/AST generated by the parser is used by later compiler phases.

Types of Parsers

Parsers are broadly divided into two categories:



Top-Down Parsing

Starts from the **start symbol** of the grammar and tries to derive the input string.

Start Symbol
↓
Non-terminals
↓
Input string

Bottom-Up Parsing

Starts with the **input tokens** and tries to reduce them to the start symbol.

Input tokens
↓
Reductions
↓
Start Symbol

---

2. Shift-Reduce Parsing

Definition

Shift-reduce parsing is a **bottom-up parsing technique** in which the parser uses a stack and input buffer to reduce the input string to the start symbol of the grammar.

It mainly performs four operations:

1. **Shift**
2. **Reduce**
3. **Accept**
4. **Error**

Basic Idea

Suppose we have:

id + id

The parser gradually processes the input and tries to convert it into the start symbol.

id + id
↓
F + id
↓
T + id
↓
E + id
↓
E + F
↓
E + T
↓
E

If the entire input becomes the start symbol `E`, the string is accepted.

Important Terms

1. Shift

Shift means moving the next input token from the input buffer onto the stack.

Example:

Input: id + id
Stack: $

After shifting `id`:

Stack: $ id
Input: + id

So:

> **Shift = Move input symbol onto stack.**

---

2. Reduce

Reduce means replacing a sequence of symbols on the stack that matches the right-hand side of a production with the corresponding left-hand side.

Suppose:

F → id

If stack contains:

$ id

we can reduce:

$ id
↓
$ F

Therefore:

> **Reduce = Replace RHS of a production by its LHS.**

---

3. Accept

When the entire input has been processed and the stack contains the start symbol, the parser accepts the input.

Example:

Stack: $ E
Input: $

Therefore:

ACCEPT

---

4. Error

If neither shifting nor reducing can produce a valid grammatical structure, a syntax error is reported.

Example:

id + * id

The parser cannot correctly process * after +.

Therefore:

ERROR

Shift-Reduce Example

Consider the grammar:

E → E + T
E → T
T → id

Input:

id + id

A simplified process is:

Stack	Input	Action

$	id + id $	Shift
$ id	+ id $	Reduce id → T
$ T	+ id $	Reduce T → E
$ E	+ id $	Shift
$ E +	id $	Shift
$ E + id	$	Reduce id → T
$ E + T	$	Reduce E + T → E
$ E	$	Accept

The important idea is:

Input → Shift → Reduce → Shift → Reduce → ... → E

Handle in Shift-Reduce Parsing

A **handle** is a substring of a sentential form that matches the **right-hand side of a production** and can be reduced to its left-hand side.

For example:

E → E + T

If we have:

E + T

then:

E + T
↓
E

Here E + T is a handle.

Important

Shift-reduce parsing repeatedly finds a **handle** and reduces it until the start symbol is obtained.

Advantages of Shift-Reduce Parsing

- Simple basic bottom-up parsing technique.
- Can handle a large class of grammars.
- Forms the basis of more powerful parsers such as **LR parsers**.
- Can handle left-recursive grammars.

Problems

- **Shift-reduce conflicts**
- **Reduce-reduce conflicts**
- Grammar may need modification.
- Basic implementation can become complex for large grammars.

---

3. Operator Precedence Parsing

Definition

Operator precedence parsing is a bottom-up parsing technique used mainly for expressions in which the precedence and associativity of operators determine how the expression should be parsed.

For example:

a + b * c

Because multiplication has higher precedence than addition:

a + (b * c)

not:

(a + b) * c

Why Operator Precedence is Needed

Consider:

a + b * c

The parser must decide:

Should it be:

(a + b) * c

or:

a + (b * c)

Operator precedence provides the answer:

* → higher precedence
+ → lower precedence

Therefore:

a + (b * c)

Operator Precedence Relations

Operator precedence parsing uses relationships between terminals.

There are three important relations:

1. a <· b

Means:

> a yields precedence to b.

The parser should generally **shift** b.

2. a =· b

Means:

> a and b have equal precedence relationship.

3. a ·> b

Means:

> a takes precedence over b.

The parser generally performs a **reduce** operation.

These relationships are stored in an **operator precedence table**.

Example

Suppose:

+ has lower precedence than *

Then:

+ < *

When the parser sees:

a + b * c

after seeing + and then *, it knows that * should be processed first.

Therefore:

a + (b * c)

Associativity

Precedence alone is not enough. We also need **associativity**.

Left Associativity

Operators such as:

+
-
*
/

are commonly left associative.

For example:

a - b - c

is interpreted as:

(a - b) - c

not:

a - (b - c)

Right Associativity

Some operators are right associative.

For example, exponentiation is commonly treated as:

a ^ b ^ c

as:

a ^ (b ^ c)

Operator Precedence Parsing Example

Expression:

id + id * id

Precedence:

* > +

The parser processes:

id + id * id

It recognizes:

id * id

first:

id + (id * id)

Then:

(id * id) → term

and finally:

id + term → expression

Conditions for Operator Precedence Grammar

Traditional operator-precedence parsing is generally applicable when the grammar satisfies restrictions such as:

1. No **ε-productions** (with the usual exception of the augmented/start handling in formulations).
2. No two adjacent non-terminals on the right-hand side.
3. Precedence relations between terminals should be determinable without ambiguity.

For example:

E → E + E

can be problematic by itself because precedence/associativity information may be insufficient unless the grammar or precedence declarations resolve it.

Advantages

- Simple for arithmetic expressions.
- Efficient for suitable expression grammars.
- Naturally handles operator precedence and associativity.
- Useful for understanding expression parsing.

Limitations

- Cannot handle every CFG.
- Grammar must satisfy specific restrictions.
- Less general than LR-family parsers.
- Mainly useful for operator-based expressions.

---

4. Top-Down Parsing

Definition

Top-down parsing is a parsing technique in which parsing starts from the **start symbol** of the grammar and tries to derive the input string by repeatedly replacing non-terminals with their productions.

In simple words:

> **Top-down parsing starts from the root of the parse tree and works toward the leaves.**

Basic Direction

Start Symbol
↓
Non-terminals
↓
Terminals
↓
Input String

For example:

```
    E
   / \
  T   + T
 / \    \
id  ...  id
```

The parser starts with E and tries to generate the input.

Example

Consider:

E → T + E | T
T → id

Input:

id + id

Start with:

E

Apply:

E → T + E

Now:

T + E

Replace T:

id + E

Replace E:

id + T

Replace T:

id + id

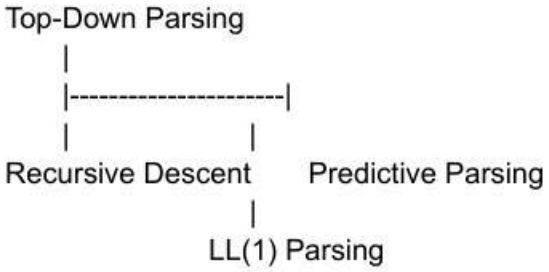
The generated string matches the input.

Therefore:

Accepted

Types of Top-Down Parsing

The major types are:



A. Recursive Descent Parsing

Definition

Recursive descent parsing is a top-down parsing technique in which each non-terminal of the grammar is generally represented by a procedure/function.

For example:

E → T + E | T
T → id

We can conceptually create:

parseE()
parseT()

The parser calls these functions recursively according to grammar productions.

Example

Grammar:

E → T + E | T
T → id

Input:

id + id

Conceptually:

parseE()
  ↓
parseT()
  ↓
id
  ↓
+
  ↓
parseE()
  ↓
parseT()
  ↓
id

Problem with Recursive Descent

A major problem is **left recursion**.

Suppose grammar is:

$E \rightarrow E + T \mid T$

A naive recursive descent parser may do:

```
parseE()
  ↓
parseE()
    ↓
parseE()
      ↓
...
```

This can result in **infinite recursion**.

Therefore, left-recursive grammar is usually transformed.

Original

$E \rightarrow E + T \mid T$

Transformed

$E \rightarrow T\ E'$
$E' \rightarrow + T\ E' \mid \varepsilon$

This form is suitable for predictive/recursive-descent parsing.

B. Predictive Parsing

Definition

Predictive parsing is a top-down parsing technique that chooses the correct production by looking at the next input token, generally without backtracking.

It is commonly associated with **LL(1) parsing**.

LL(1)

The name means:

- First **L** = scan input from **Left to right**
- Second **L** = produce a **Leftmost derivation**

- **1** = use **one lookahead token**

So:

> **LL(1) parser reads input from left to right, constructs a leftmost derivation, and uses one lookahead symbol.**

Example of Predictive Parsing

Grammar:

E → T E'
E' → + T E' | ε
T → id

Input:

id + id

The parser looks at the next token and decides which production to use.

E
↓
T E'
↓
id E'
↓
id + T E'
↓
id + id E'
↓
id + id

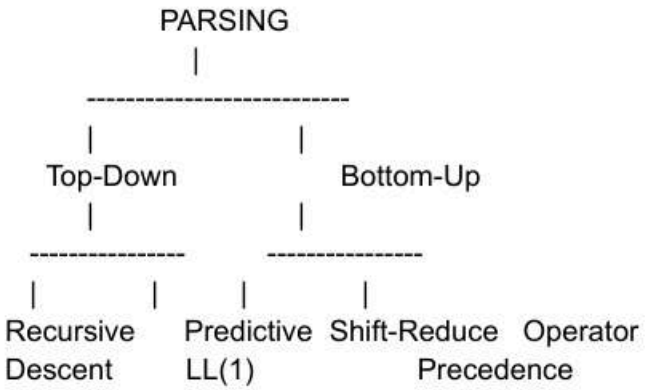
The input is successfully parsed.

Top-Down vs Bottom-Up Parsing

Feature	Top-Down	Bottom-Up
Starting point	Start symbol	Input tokens
Direction	Root → leaves	Leaves → root

Derivation	Leftmost derivation	Reverse of rightmost derivation
Main idea	Generate input	Reduce input
Example	Recursive Descent, LL(1)	Shift-Reduce, LR
Left recursion	Problematic	Can handle it
Parse tree	Built from root	Built from leaves

All Four Topics Together



Top-Down

Start Symbol
↓
Rules
↓
Input

Bottom-Up

Input
↓
Shift
↓
Reduce
↓
Start Symbol

🔥 Important Exam Differences

Point	Shift-Reduce	Operator Precedence	Top-Down

Type	Bottom-up	Bottom-up	Top-down
Starts with	Input	Input	Start symbol
Main operation	Shift & Reduce	Precedence-based shift/reduce	Derivation
Common use	General bottom-up parsing	Expressions	General/simple grammars
Example	LR family foundation	Arithmetic expressions	Recursive descent, LL(1)
Main issue	Conflicts	Grammar restrictions	Left recursion/backtracking

1. Predictive Parsers

A **predictive parser** is a type of **top-down parser** that predicts which grammar production should be used by looking at the next input token.

It constructs the parse tree starting from the **start symbol** and proceeds toward the input string.

The important point is:

> A predictive parser chooses the correct production **without backtracking**.

Basic flow

Input Tokens
↓
Lookahead Token
↓
Parsing Table
↓
Select Production
↓
Expand Non-terminal
↓
Continue Parsing

Example

Consider:

E → T E'
E' → + T E' | ε
T → id

Input:

id + id

The parser starts with:

E

Using:

E → T E'

we get:

T E'

Then:

T → id

so:

id E'

When the next input is +, the parser chooses:

E' → + T E'

Therefore:

id + T E'

Then:

T → id

giving:

id + id E'

Finally:

E' → ε

So the input is accepted.

LL(1) Predictive Parser

The most common predictive parser is **LL(1)**.

Meaning of LL(1)

L → Left-to-right scanning
L → Leftmost derivation
1 → One lookahead symbol

Therefore:

> An LL(1) parser reads the input from left to right, produces a leftmost derivation, and uses one lookahead token.

FIRST and FOLLOW

Predictive parsing tables are generally constructed using **FIRST** and **FOLLOW** sets.

FIRST

`FIRST(X)` contains the terminals that can appear at the beginning of strings derived from `X`.

Example:

E → T E'
T → id

Therefore:

FIRST(T) = { id }

FOLLOW

`FOLLOW(A)` contains terminals that can appear immediately after non-terminal `A`.

For example, if:

E → T E'

then symbols that can follow `E'` are considered when constructing the table.

Advantages

- Simple to understand and implement.
- Does not require backtracking for LL(1) grammars.
- Efficient parsing.
- Parsing table is relatively easy to construct.

Limitations

- Not every CFG can be directly used.
- Left-recursive grammars must be transformed.
- Sometimes **left factoring** is required.

---

2. Automatic Construction of Efficient Parsers

Manually constructing a parser for a large programming language is difficult.

Therefore, compiler theory provides **parser generators**.

Grammar
↓
Parser Generator
↓
Parsing Tables
↓
Parser

The most important automatically constructed parsers are:

LR Parsers
|
|--- SLR
|
|--- Canonical LR / CLR
|
|--- LALR

These are **bottom-up parsers**.

---

3. LR Parsers

Definition

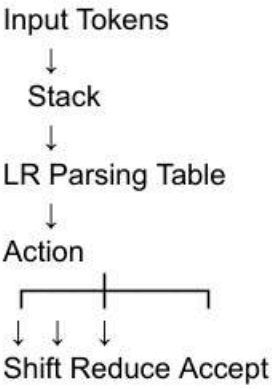
An **LR parser** is a bottom-up parser that scans the input from **Left to right** and constructs a **Rightmost derivation in reverse**.

LR means

L → Left to right scanning
R → Rightmost derivation in reverse

LR parsers are powerful because they can handle a large class of context-free grammars.

Basic LR Parsing Process



The LR parser uses two tables:

ACTION table

Determines what action should be performed.

Possible actions:

shift
reduce
accept
error

GOTO table

Determines the next state after a non-terminal is pushed.

Example

Suppose:

S → CC
C → cC
C → d

Input:

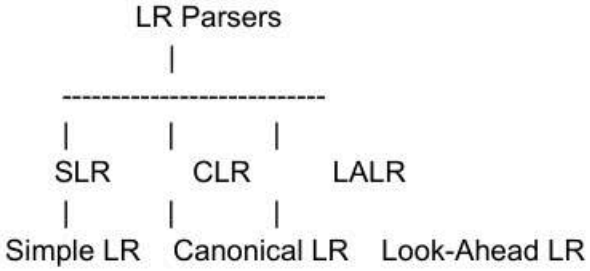
cdd

An LR parser processes the input using states and parsing-table actions.

Conceptually:

Input
↓
Shift
↓
Reduce
↓
Shift
↓
Reduce
↓
Accept

Types of LR Parsers



Generally:

SLR < LALR < CLR

in terms of grammar-handling power, while table size generally follows:

SLR < LALR < CLR

---

4. Canonical Collection of LR(0) Items

This is one of the **most important concepts** for constructing LR parsing tables.

What is an LR(0) Item?

An **LR(0) item** is a production with a dot · showing how much of the production has been recognized.

Suppose:

$A \rightarrow XYZ$

Its LR(0) items are:

$A \rightarrow \cdot XYZ$
$A \rightarrow X \cdot YZ$
$A \rightarrow XY \cdot Z$
$A \rightarrow XYZ \cdot$

The dot tells us the current position of the parser.

Meaning of the Dot

Consider:

$A \rightarrow X \cdot Y\ Z$

This means:

> X has already been recognized, and the parser expects Y next.

If:

$A \rightarrow X\ Y \cdot Z$

then:

> X and Y have been recognized, and Z is expected next.

If:

$A \rightarrow XYZ \cdot$

the dot is at the end.

This is called a **completed item**.

Augmented Grammar

Before constructing LR(0) items, we augment the grammar.

Suppose:

S → CC

We create:

S' → S
S → CC

The new start production is called the **augmented production**.

The purpose is to provide a unique acceptance condition:

S' → S ·

means:

ACCEPT

Closure

Definition

The **closure** operation adds all items that may be needed when a dot appears before a non-terminal.

Suppose we have:

A → α · B β

and:

B → γ
B → δ

Then closure adds:

B → ·γ
B → ·δ

If those newly added items have dots before other non-terminals, we continue adding their productions until no more items can be added.

Example

Grammar:

S → CC
C → cC
C → d

Start with:

S' → ·S

Since dot is before S:

S → CC

is added:

S' → ·S
S → ·CC

Now dot is before C, so add:

C → ·cC
C → ·d

Therefore:

I0 =
S' → ·S
S → ·CC
C → ·cC
C → ·d

This is the first LR(0) state.

GOTO

Definition

`GOTO(I, X)` tells us what state we reach from item set `I` after recognizing grammar symbol `X`.

Suppose:

$A \rightarrow \alpha \cdot X \beta$

Move the dot over `X`:

$A \rightarrow \alpha X \cdot \beta$

Then take closure.

Example

If:

I0:
$C \rightarrow \cdot cC$

then:

GOTO(I0, c)

moves the dot:

$C \rightarrow c \cdot C$

Since the dot is before `C`, add:

$C \rightarrow \cdot cC$
$C \rightarrow \cdot d$

So the new state contains:

$C \rightarrow c \cdot C$
$C \rightarrow \cdot cC$
$C \rightarrow \cdot d$

Canonical Collection

The **canonical collection of LR(0) items** is the complete set of item sets obtained by repeatedly applying:

CLOSURE
GOTO

starting from the augmented start item.

Process

Augmented Grammar
↓
Initial Item
↓
CLOSURE
↓
I0
↓
GOTO
↓
I1, I2, I3...
↓
Continue until no new states

These item sets become the **states of the LR automaton**.

---

5. Constructing SLR Parsing Tables

SLR

SLR stands for:

Simple LR

An SLR parser uses:

1. LR(0) item sets
2. FOLLOW sets

to construct the parsing table.

SLR Table Structure

There are two parts:

ACTION
GOTO

ACTION

Columns correspond to terminals:

id + * ( ) $

GOTO

Columns correspond to non-terminals:

E T F

Rules for Constructing SLR Table

Suppose state `Ii` contains:

$A \rightarrow \alpha \cdot a \beta$

where `a` is a terminal.

If:

GOTO(Ii, a) = Ij

then:

ACTION[i, a] = shift j

Reduce Rule

Suppose state `Ii` contains:

$A \rightarrow \alpha \cdot$

The item is complete.

For every terminal:

a ∈ FOLLOW(A)

put:

ACTION[i,a] = reduce A → α

Accept Rule

If state contains:

S' → S ·

then:

ACTION[i,$] = ACCEPT

GOTO Rule

If:

GOTO(Ii, A) = Ij

where A is a non-terminal:

GOTO[i,A] = j

Why FOLLOW is Used in SLR

Suppose:

A → α ·

We cannot reduce on every terminal.

Instead, we reduce only when the lookahead belongs to:

FOLLOW(A)

This helps prevent unnecessary reductions.

---

6. Constructing Canonical LR Parsing Tables

Canonical LR is also called:

CLR — Canonical LR

It is more powerful than SLR.

The major difference is:

SLR → LR(0) items + FOLLOW
CLR → LR(1) items + lookahead

LR(1) Item

An LR(1) item has:

[A → α · β, a]

where:

- `A → α · β` is the LR(0) item.
- `a` is the **lookahead symbol**.

Example:

[A → α · β, $]

means that `$` is a possible lookahead.

Why Lookahead is Important

SLR uses the same FOLLOW set for all occurrences of a production.

CLR uses a **specific lookahead for each item**.

Therefore, CLR can make more precise shift/reduce decisions.

Closure for LR(1)

Suppose:

$[A \rightarrow \alpha \cdot B \beta, a]$

and:

$B \rightarrow \gamma$

Then add:

$[B \rightarrow \cdot\gamma, b]$

where:

$b \in FIRST(\beta a)$

This is more precise than ordinary LR(0) closure.

CLR Table Construction

The process is:

Augmented Grammar
↓
Construct LR(1) items
↓
CLOSURE
↓
GOTO
↓
Canonical LR(1) collection
↓
ACTION + GOTO
↓
CLR Parsing Table

CLR Advantages

- More powerful than SLR.
- Uses explicit lookahead.

- Resolves some conflicts that SLR cannot.
- Can recognize more grammars.

Disadvantage

The number of states can become very large.

Therefore:

Large table
More memory
More construction complexity

---

7. Constructing LALR Parsing Tables

LALR

LALR stands for:

> **Look-Ahead LR**

It tries to combine the advantages of SLR and CLR.

The basic idea is:

> Construct CLR states and then merge states having the same LR(0) core.

What is LR(0) Core?

Consider these LR(1) items:

$[A \rightarrow \alpha \cdot \beta, a]$
$[A \rightarrow \alpha \cdot \beta, b]$

The LR(0) core ignores the lookahead:

$A \rightarrow \alpha \cdot \beta$

If two states have the same LR(0) cores, they can generally be merged in LALR construction by combining their lookaheads.

Example

Suppose we have two CLR states:

I5:
$A \rightarrow \alpha \cdot \beta, a$
$B \rightarrow \gamma \cdot, b$

and:

I8:
$A \rightarrow \alpha \cdot \beta, c$
$B \rightarrow \gamma \cdot, d$

Their LR(0) cores are:

$A \rightarrow \alpha \cdot \beta$
$B \rightarrow \gamma \cdot$

Therefore they can be merged:

I5/I8:
$A \rightarrow \alpha \cdot \beta, a/c$
$B \rightarrow \gamma \cdot, b/d$

LALR Construction Process

Grammar
↓
Construct LR(1) states
↓
Find states with same LR(0) core
↓
Merge their lookaheads
↓
Construct ACTION/GOTO table
↓
LALR Parser

Why LALR is Popular

LALR provides:

- Much smaller tables than CLR.
- More power than SLR.
- Good practical balance.

LALR parsers have historically been widely used in compiler tools such as **YACC-style parser generators**.

SLR vs CLR vs LALR

Feature	SLR	CLR	LALR
Full name	Simple LR	Canonical LR	Look-Ahead LR
Items	LR(0)	LR(1)	LR(1) merged by LR(0) core
Lookahead	FOLLOW(A)	Explicit	Explicit, after merging
States	Small	Large	Usually close to SLR
Power	Lower	Highest of these three	Between SLR and CLR
Table size	Small	Large	Medium
Complexity	Low	High	Medium
Practical use	Educational/useful	More powerful but larger	Very common

A useful memory line:

SLR → Simple
CLR → Complete/Canonical
LALR → CLR states merged to become smaller

---

8. Using Ambiguous Grammars

What is an Ambiguous Grammar?

A grammar is **ambiguous** if at least one string has more than one parse tree.

Example:

E → E + E
E → E * E
E → id

Consider:

id + id * id

It can be interpreted as:

(id + id) * id

or:

id + (id * id)

Therefore, the grammar is ambiguous.

Why Ambiguity is a Problem

A parser needs to know exactly what structure the program has.

For example:

a + b * c

Normally we want:

a + (b * c)

because * has higher precedence.

An ambiguous grammar does not inherently specify this.

Resolving Ambiguity

There are two common approaches.

1. Rewrite the grammar

Instead of:

E → E + E

E → E * E
E → id

use:

E → E + T | T
T → T * F | F
F → id

This grammar establishes:

* > +

---

2. Use precedence and associativity declarations

Parser generators can sometimes resolve conflicts by specifying:

* → higher precedence
+ → lower precedence

and:

+ → left associative
* → left associative

This allows an otherwise ambiguous expression grammar to be handled using parser-generator rules.

Shift-Reduce Conflict

Ambiguous grammars often cause:

Shift/Reduce Conflict

The parser cannot decide whether to:

SHIFT

or:

REDUCE

For example:

id + id * id

At a particular point, the parser may need to decide whether to reduce `id + id` or shift `*`.

Operator precedence can resolve this.

---

9. An Automatic Parser Generator

Definition

An **automatic parser generator** is a software tool that takes a formal grammar and generates a parser automatically.

Instead of manually programming all parser states and actions, we provide:

Grammar
+
Semantic Actions
+
Token Information

and the tool generates the parser.

Basic Architecture

Grammar Specification
↓
Parser Generator
↓
Parsing Tables
↓
Generated Parser
↓
Input Tokens
↓
Syntax Analysis

Example: YACC

YACC stands for:

Yet Another Compiler-Compiler

It is a parser generator traditionally used to generate parsers for programming languages.

A YACC-style specification generally contains:

Declarations
%%
Grammar Rules + Semantic Actions
%%
Supporting Code

LEX + YACC

In a compiler, lexical and syntax analysis can work together:

Source Code
↓
LEX
↓
Tokens
↓
YACC
↓
Parser
↓
Parse Tree / Semantic Actions

LEX

Handles:

Characters → Tokens

YACC

Handles:

Tokens → Syntax

---

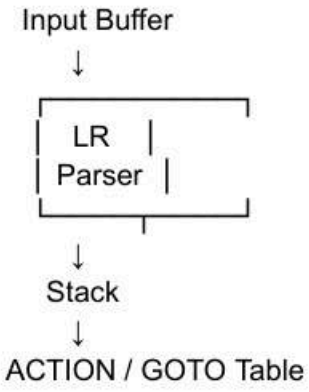
10. Implementation of LR Parsing Tables

Once an LR parsing table has been constructed, we need to implement a program that actually uses that table to parse input.

The LR parser mainly maintains:

1. **Stack**
2. **Input buffer**
3. **ACTION table**
4. **GOTO table**

LR Parser Structure



Stack

The stack usually contains **states**, and in conceptual descriptions may also be shown with grammar symbols.

Example:

$ 0

After shifting a token:

$ 0 id 5

Here:

id → grammar symbol
5 → state

Input Buffer

The input buffer contains tokens followed by an end marker:

id + id $

$ indicates the end of input.

ACTION Table

The ACTION table tells the parser what to do for a particular:

(state, lookahead)

Possible actions:

Shift
Reduce
Accept
Error

For example:

ACTION[5, +] = shift 7

means:

> If the parser is in state 5 and the next input symbol is +, shift + and move to state 7.

GOTO Table

GOTO is used after a reduction.

Example:

GOTO[3, E] = 8

means:

> After reaching state 3 and recognizing non-terminal E, move to state 8.

LR Parsing Algorithm

The basic algorithm is:

Step 1

Initialize stack:

$ 0

Step 2

Look at:

Current state
+
Next input token

Step 3

Look up:

ACTION[state, token]

Step 4

Perform the action.

If Action = Shift

Suppose:

ACTION[0,id] = shift 5

Then:

Push id
Push state 5
Move input pointer

If Action = Reduce

Suppose:

$A \rightarrow \beta$

and parser says:

reduce A → β

Then:

1. Remove symbols/states corresponding to β.
2. Look at the state now on top.
3. Push A.
4. Use GOTO to determine the new state.

For example:

Stack:

$ 0 X 3 Y 7

Reduction:
Y → A

After removing Y/state:
$ 0 X 3

Push A:
$ 0 X 3 A

GOTO(3,A) = 8

Final:
$ 0 X 3 A 8

If Action = Accept

ACTION[state,$] = ACCEPT

Then:

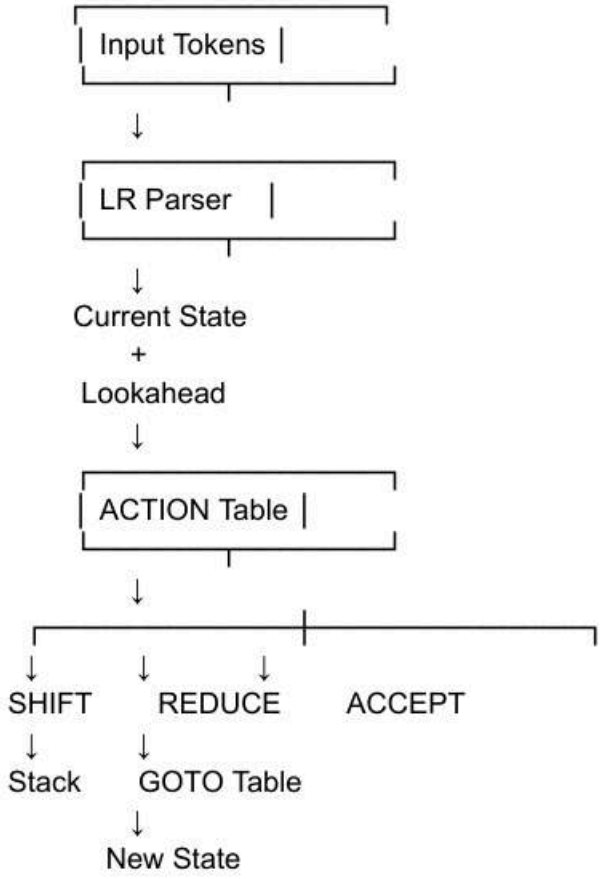
Input is syntactically valid.

If Action = Error

The parser reports:

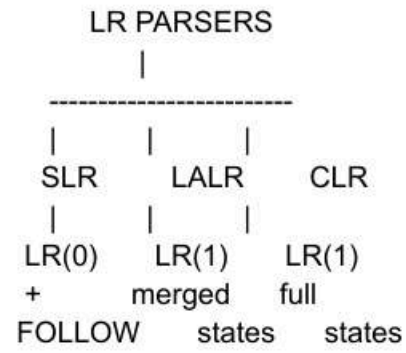
Syntax Error

Complete LR Parsing Process



🔥 Most Important Concept: SLR vs CLR vs LALR

Remember this progression:



SLR

LR(0) items + FOLLOW

CLR

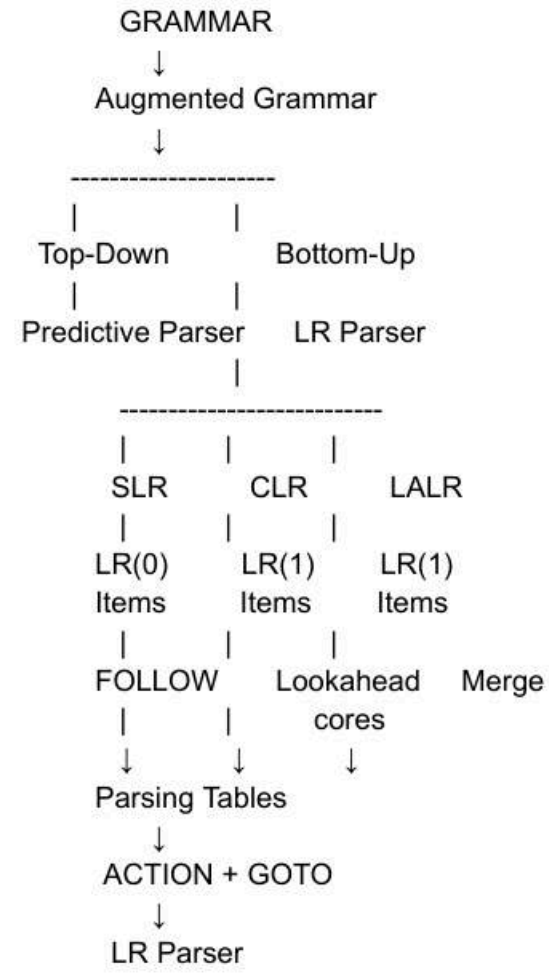
LR(1) items + explicit lookahead

LALR

CLR states
↓
Same LR(0) cores
↓
Merge states

⭐ Complete Unit Flow

The entire topic can be remembered as:



📝 Exam-Ready Definitions

Predictive Parser

A predictive parser is a top-down parser that selects the appropriate production using the next input symbol without backtracking.

LR Parser

An LR parser is a bottom-up parser that scans input from left to right and constructs a rightmost derivation in reverse.

LR(0) Item

An LR(0) item is a production with a dot indicating how much of the production has already been recognized.

Canonical Collection of LR(0) Items

It is the complete collection of LR(0) item sets obtained by repeatedly applying closure and goto operations to the augmented grammar.

SLR Parser

SLR (Simple LR) uses LR(0) items and FOLLOW sets to construct ACTION and GOTO parsing tables.

CLR Parser

CLR (Canonical LR) uses LR(1) items containing explicit lookahead symbols to construct parsing tables.

LALR Parser

LALR (Look-Ahead LR) is constructed by merging CLR states having the same LR(0) core, resulting in smaller parsing tables.

Ambiguous Grammar

A grammar is ambiguous if at least one string generated by it has more than one parse tree.

Automatic Parser Generator

An automatic parser generator is a tool that takes a grammar specification and automatically generates a parser and its parsing tables.

LR Parsing Table

An LR parsing table consists of ACTION and GOTO tables used by the LR parser to determine shift, reduce, accept, and state-transition operations.

🧠 One-line memory trick

Predictive → Predict the production
LR → Bottom-up parsing
LR(0) → Dot only
SLR → LR(0) + FOLLOW
CLR → LR(1) + Lookahead
LALR → CLR + merge same cores
ACTION → What action to perform
GOTO → Which state to go to